

AARHUS UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE

COMPILATION

Hand-in 2

Author

Søren M. DAMSGAARD

Student number

202309814

September 14, 2026



Expression Programs

Task 1

It uses a lot of the same logic as the previous pretty_printer from the first assignment.

The main function `string_of_eprog` uses the `fold_left` list function to iterate over all the statements and letting a helper function evaluate them and then append their variable to their statement, printing them on their own line.

Semantic Analysis

Task 2

When checking the semantics, we go through the whole syntax tree and check for undeclared and duplicate variables if any errors are found, the compilation will be stopped at the end and a list of errors are presented in order of their happening.

Q1

Yes, `semant` needs to be recursive, although the function in itself isn't recursive, it uses the functions `aux` and `c_expr` to loop through the math in all the statements. `aux` works great here since it's tail-recursive and `c_expr` is recursive since it needs to go through nested operations

Q2

Variable declarations are tracked using the `string list`, which acts as the environment. As we go through statements, the `List.mem` check if some elements already exist in the list, which is used for catching both duplicates and undeclared variables.

Q3

Glory Question - Not Implemented

Evaluator

Task 3

The evaluator takes the verified/validated expression programs and folds over the statements to create an environment, and with the help of `eval_expr` it evaluates it all and returns the result of the final expression

Q4

If we use only 'let' the function could be written as

```
let () = Printf.printf "Please enter an integer: " in let line = read_line ()
in int_of_string line
```

Q5

Glory Question - Not Implemented

Q6

The interpreter uses an associative list type `(string * int) list` which essentially functions like a map, that can be looked up throughout the program.

Q7

Right now a classic division by zero would crash the whole program, since no step has been taken to prevent it, although a statement in the `eval_expr` could possibly fix this by letting it throw an error.

Compiling to x86 Assembly

Task 4

The compiler translates the AST into x86 assembly using the stack-spilling strategy. Local variables and temporary values are assigned an 8-byte slot on the stack, and math operations are executed by moving these values into and out of the `%rax` register.

Note this part of the code the `compile.ml` is vibe-coded hence the description here might not be adequate.

Q8

Glory Question - Not Implemented

Q9

As mentioned this is not wholly my original work, I went with the associative list again, from the interpreter, to map variables to their stack layout offset, but the use of `int ref` to handle mutables/temporaries and standard lists to build the final x86 code wasn't my idea.

Q10

This is straight up just pilling everything on the stack and hoping it works, this tends to make things slow compared to using the CPU's registers.

Q11

Glory Question - Not Implemented

Task 5

The wrapper functions, `interpret` and `compile`, in the main file, first run the semantic analysis to check for errors. If the semantic check returns `Ok`, it either runs the evaluator to get the final integer, or it generates the x86 assembly and writes it to a file `program.s`. This way it can be cross-tested by feeding the exact program into both of them and checking the output of the evaluator and the x86 assembly + C code.

Q12

There is only one example program that test the basics, variable binding and addition.

How to run the code

You will have received a zip file containing the complete OCaml compiler source code, the Dune build configurations, and the C runtime environment.

This is run/compiled in VScode using the docker container provided.

To compile and run the project inside a VS Code development container, execute the following commands in the terminal while inside the folder:

1. `dune build`
2. `dune exec ./main.exe` (this runs the interpreter and compile functions)
3. `clang main.c program.s -o program`
4. `./program`